PA-02 Viva Preparation Guide

Re-engineering the ls Utility — BSDSF24A040-OS-A02

BSDSF24A040 — Operating Systems, Programming Assignment 02

Repository: https://github.com/GulfamRasheed/BSDSF24A040-OS-A02 **Total marks:** 100 · **Viva weight:** the difference between a good grade and a zero

> **The rule that governs everything below.** The brief states: *"Do not blindly copy-paste code from AI. You are expected to understand the 'how' and 'why' of every line you submit, and failure to demonstrate this to a TA will result in a zero for the assignment."*
>
> So every answer here is written as **what to say**, not just **what is true**. If you can say these out loud without reading, you are ready.

Part 1 — The 60-second pitch

Memorise this. It is almost always the opening question.

> "I re-implemented the Unix `ls` command in C, starting from the instructor's `lsv1.0.0` base code and extending it across seven tagged versions. The final version supports `-l` long listing, `-x` horizontal columns, `-R` recursion, alphabetical sorting, and ANSI colour output by file type.
>
> The design principle throughout was to **separate data preparation from data presentation**. `do_ls()` reads the directory into a dynamically allocated array, sorts it once, and *then* dispatches to one of three display functions. That is why adding `-R`, `-x` and colour required zero changes to the long-listing code, and why all three display modes come out sorted without any of them knowing that sorting exists.
>
> System calls used: `opendir`, `readdir`, `closedir`, `lstat`, `readlink`, `getpwuid`, `getgrgid`, `ctime`, `ioctl` with `TIOCGWINSZ`, `qsort`, `strdup`, `realloc`, `isatty`, `snprintf`.
>
> Everything is built through a Makefile with zero warnings under `-Wall -Wextra`, and I validated it with 75 runs under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer — zero findings."

If asked for one sentence: *"The whole design is one function that prepares data and three that render it, and keeping those separate is why each feature cost so little code."*

Part 2 — System calls you must know cold

If a TA asks "explain this call", you must be able to answer in one breath.

Call	Signature	Returns	What it actually does
`opendir`	`DIR *opendir(const char *)`	stream handle or `NULL`	Asks the kernel for a directory stream
`readdir`	`struct dirent *readdir(DIR *)`	entry or `NULL`	Next entry; `d_name` **points into a buffer reused every call**
`closedir`	`int closedir(DIR *)`	0 / -1	Releases the stream
`lstat`	`int lstat(const char *, struct stat *)`	0 / -1	Metadata **without** following symlinks
`stat`	`int stat(const char *, struct stat *)`	0 / -1	Metadata **following** symlinks
`readlink`	`ssize_t readlink(const char *, char *, size_t)`	bytes or -1	Reads a symlink target; **does not NUL-terminate**
`getpwuid`	`struct passwd *getpwuid(uid_t)`	record or `NULL`	uid → username, from `/etc/passwd`
`getgrgid`	`struct group *getgrgid(gid_t)`	record or `NULL`	gid → group name, from `/etc/group`
`ctime`	`char *ctime(const time_t *)`	26-byte string	`"Tue Mar  5 14:30:07 2024"`
`ioctl`	`int ioctl(int, unsigned long, ...)`	0 / -1	Device control; `TIOCGWINSZ` → terminal size
`isatty`	`int isatty(int)`	1 / 0	Is fd 1 a real terminal?

Three traps the examiner will probe

1. `readdir()` returns `NULL` for two different reasons. > "End of directory" *and* "error". So you must set `errno = 0` before the > loop and inspect `errno` after `readdir()` returns `NULL`. Without that > you cannot tell success from failure. The base code does exactly this.

2. `readdir()` reuses one buffer. > `entry->d_name` is a pointer into an internal buffer that the *next* call > overwrites. Storing that pointer in an array would make every slot alias > the last name read. This is exactly why we `strdup()`.

3. `getpwuid()` / `getgrgid()` return a static buffer. > The next call overwrites it. You must never `free()` it, and you must > copy the string out if it needs to outlive the next call. We print > immediately, so direct use is safe.

Part 3 — The ten report questions, condensed

These are the exact questions in `REPORT.md`. The full answers are there; this is the part you must recall.

Q1 — `stat()` vs `lstat()`

One line: *Both fill the same `struct stat`; the only difference is whether symbolic links are followed.*

	`stat()`	`lstat()`
Symlink to a regular file	metadata of the **target**	metadata of the **link**
`st_mode` has `S_IFLNK`?	no	**yes**

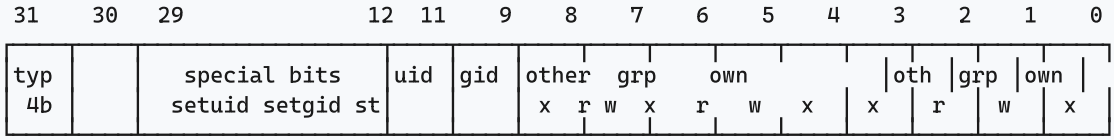
When `ls` needs `lstat()`: - `ls -l` must print `l` in the first permission column and `link -> target`. That information exists **only** in the link's own inode — `stat()` discards it. - `ls -R` must use it before recursing. If a subdirectory contains a symlink pointing back up towards the root, `stat()` reports "directory" and we follow it into an **infinite cycle**. `lstat()` reports "symlink" and we correctly do not descend.

The mirror case where `stat()` is right: if you genuinely want the size of the data — for `/usr/bin/gcc -> gcc-13` it is the size of `gcc-13` the user cares about, not the 10-byte link.

> **Cost note if asked:** `stat()` internally does `lstat()` and then repeats the lookup on whatever the link resolved to, so it is strictly more expensive.

Q2 — Extracting type and permission bits with `&`

The premise: `st_mode` packs three disjoint fields into one integer, so each can be extracted with an AND against a mask.



Key idea: `mode & MASK` yields the masked bits when set and `0` when clear, so the result works directly as a boolean. *An AND with a one-bit mask is a bit test in disguise.*

Type — mask then switch

```c
switch (mode & S_IFMT) {          /* S_IFMT = 0170000, all four type bits */
    case S_IFDIR: t = 'd'; break;
    case S_IFREG: t = '-'; break;
    ...
}
```

> `S_IFMT` **is the crucial detail.** Masking with it keeps *only* the type bits so the equality test is not polluted by permission bits. Testing `mode & S_IFDIR` directly is wrong for every other type — a directory's mode `040755 & 0100000` is non-zero, yet the file is not a regular file.
>
> The instructor's own `filetype.c` uses the equivalent form `(mode & 0170000) == 0010000` — `0170000` **is** `S_IFMT`.

Permissions — one loop covers all nine

```
const char *rwx = "rwxrwxrwx";
for (int i = 0; i < 9; i++)
    out[i] = (mode & (1 << (8 - i))) ? rwx[i] : '-';
```

> The shift direction is the only subtlety: bit 8 is *owner read* → `rwx[0]`; bit 0 is *other execute* → `rwx[8]`. Hence `8 - i`.

Special bits override the execute character

```
if (mode & S_ISUID) out[2] = (mode & S_IXUSR) ? 's' : 'S';
```

> They share a bit position with `x`. GNU `ls` uses lower case when the execute bit is **also** set, upper case when it is not: setuid+executable → `s`, setuid-only → `S`. The *first* `&` isolates the setuid bit; the *second* distinguishes the two letter cases.

Q3 — Down-then-across: the algorithm, and why one loop fails

```
col_width = longest_filename + 2
num_cols  = terminal_width / col_width
num_rows  = (N + num_cols - 1) / num_cols        /* ceiling division */

for r = 0 .. num_rows-1:          /* ROW first    */
    for c = 0 .. num_cols-1:      /* COLUMN second */
        idx = c * num_rows + r
        if idx < N: print names[idx] padded to col_width
    newline
```

The one line that carries the whole idea: `idx = c * num_rows + r`. A column holds `num_rows` items, so reaching column `c` means adding `c * num_rows` to the base offset of row `r`.

Ceiling division: `(a + b - 1) / b` rounds `a/b` up. Needed so leftovers land in the *first* columns and the last column is short — exactly what GNU `ls` does, with no special case.

Why one loop is not enough

```
row-major (wrong)        column-major (correct)
a b c d e                a c e
f g h i j                b d f
                         g i j
                         h
```

1. `num_rows` **must be known before printing starts.** `num_rows` depends on `num_cols`, which depends on the longest name *and* the terminal width.
2. **The terminal can be resized.** The mapping is `c = i / num_rows`, `r = i % num_rows`. Change the width and `num_rows` changes, so the *entire* index-to-position mapping changes. In row-major it is trivially stable — which is precisely why it is the wrong layout.
3. **Ragged edges.** Early columns can be one row longer.

> **One sentence summary:** down-then-across is a *transposition* of the input order, so the index-to-position map is not linear and you need a nested loop.

Q4 — `ioctl()`, and the cost of a fixed fallback

```
struct winsize ws;
if (ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws) == -1 || ws.ws_col == 0)
    return 80;                          /* fallback */
return ws.ws_col;
```

What it does: `ioctl()` is the generic device-control entry point; a terminal driver is just another device. `TIOCGWINSZ` ("Terminal I/O Get Window Size") tells the driver which piece of information we want; it fills `struct winsize`, whose `ws_col` holds the column count.

Why fd 1 specifically: that is the stream the output is destined for — the width that matters is the width of wherever the text is going.

Why not the alternatives: - `system("tput cols")` → forks a shell, execs `tput`, parses output, on **every** invocation of a tool meant to be instant. - `$COLUMNS` → not exported by default, and can be stale after a resize. - Hard-coded 80 → loses all responsiveness.

Limitations of a fixed 80-column fallback

1. **Horizontal overflow.** In a 200-column terminal an 80-wide layout wastes 120 columns — a **60 % loss** of usable screen.
2. **Layout corruption.** In a 40- or 60-column terminal the columns are wider than the window, the terminal performs its own soft wrap, and two logical rows fuse into one visual row. The grid is no longer aligned.
3. **No embedded support.** Wrong in an SSH session on a phone, a split `tmux` pane, an editor's embedded terminal, a `less` preview.
4. **Lost density.** For short names, 80 columns gives 14 columns when the real terminal shows 20 at 120 and 40 at 200.
5. **Silently wrong, not an error.** The fixed path always *produces* output, so nothing reports the problem — the worst kind of failure for a system utility.

> **Measured proof from our own testing:** the column count tracked the terminal width exactly — 40→8 columns, 60→10, 80→14, 120→20, 200→40.

Q5 — Vertical vs horizontal complexity

Down-then-across needs substantially more pre-computation, for one reason: it is a **transposition**.

- Its mapping is `(row, col) = (i % num_rows, i / num_rows)` — the **modulo is on the row**, so every item needs `num_rows`, which needs `num_cols`, which needs both the longest name *and* the terminal width. All of it must be resolved **before the first byte is printed**.
- Across has mapping `(row, col) = (i / num_cols, i % num_cols)` — division on the column. It depends only on values already in hand, so nothing waits on anything.

	down-then-across	across
Pre-computed	`col_width`, `num_cols`, `num_rows`	`col_width`, `num_cols`
Loop	nested (row outer, column inner)	single linear loop
Index arithmetic	`c * num_rows + r` — multiply + add	`i % num_cols` — modulo only
Row count needed?	**yes**	no

Q6 — Display-mode state management

```
typedef enum { MODE_COLUMN, MODE_LONG, MODE_ACROSS } DisplayMode;
static DisplayMode display_mode = MODE_COLUMN;
```

Why not three booleans. The modes are mutually exclusive, and booleans cannot express "neither of the two things I have flags for". With two booleans you need a *fourth* variable plus a priority rule to decide which wins when both are set. An enum makes the state space explicit and makes a contradictory combination unrepresentable.

Three-stage flow: 1. `getopt()` populates the state — the optstring `"lxR"` declares which letters are valid; a `:` after a letter would mean it takes an argument. 2. `do_ls()` builds the array once, sorts once, then dispatches with a single `if / else if`. 3. All three modes receive the *same* `NameList`.

`-R` is deliberately a separate boolean, not a fourth enum value. Recursion is *orthogonal* to display mode: `-lR` must mean long listing **and** recursion at once. As an enum value it would need eight combinations, five of them meaningless. Separating them keeps it at 3 × 2.

Known simplification: flag order decides the winner, so `ls -l -x` ends up horizontal. Real GNU `ls` rejects the combination.

Q7 — Why read everything into memory before sorting

Sorting is not a property of a file; it is a *relation between* files. So you must be able to compare any two. `readdir()` gives no such access:

- It returns entries **one at a time**, and `d_name` points into a single buffer the next call overwrites.
- The stream is **forward-only** — no seek, no rewind, no "go back to entry 3".
- The kernel returns entries in **filesystem order** — inode order on ext4, hash order on XFS — not a meaningful one.

Materialising the names is the only place sorting *can* happen. *(GNU `ls` solves the same problem differently: it also reads the whole directory, but if the result is too large it re-opens and sorts in bounded chunks, merging them — merge sort instead of `qsort`.)*

Drawbacks at millions of files

1. **Memory.** Each entry costs a `strdup()` (~32 bytes allocator overhead plus the name) **and** an 8-byte pointer. At 10 million entries that is roughly **400 MB**. Doubling `realloc` means a transient peak of ~1.5 *n* pointers.

2. **Peak = all of it, at once.** It cannot be freed early — the longest name and the total count are only known after the last entry, and both are needed for the layout.
3. **Latency.** Nothing prints until the last entry is read. On a network filesystem with 200 ms latency, a large directory *feels frozen*, whereas real `ls` streams progressively.
4. **No streaming fallback.** A directory that genuinely cannot fit in RAM cannot be listed at all.
5. **Sort cost.** `qsort` is O(*n* log *n*) with no guaranteed bound.
6. **Recursion multiplies it.** Under `-R`, every level would hold its own array. **We free each level before recursing**, so at most one level's worth is ever live.

Q8 — The `qsort()` comparison function

```
int cmp_names(const void *a, const void *b);   /* forced by the standard */
```

The signature is fixed by C because `qsort` must call a function whose argument types it cannot know — it sorts a byte range of unknown type.

What `qsort` actually passes

`qsort` sorts **elements**, and hands us the **addresses** of two of them. Our elements are `char *`, so the cast happens **twice**:

```
a  -->  (char **)    points AT our char * slot
         |
         +-->  (char *)   dereference to reach the string itself
```

```
const char *sa = *(const char *const *)a;
const char *sb = *(const char *const *)b;
return strcmp(sa, sb);
```

> **Getting one dereference wrong is the classic bug** — you end up comparing *pointer addresses* instead of strings.

Why `const void *`

- `void *` — `qsort` is type-agnostic by design; it has no idea whether the array holds `int`, `double` or `char *`, so it can only pass a generic pointer.
- `const` — `qsort` **sorts in place** but never modifies the *elements*; it only moves them. `const` states that guarantee, prevents accidental writes, and lets the function be used on a genuinely `const` array.

The return contract

negative if `*a` precedes `*b`, zero if equal, positive if it follows.

> **Never return `(a - b)`** — arithmetic on unrelated addresses is undefined behaviour and can overflow. `strcmp()` already returns exactly the value we need.

Q9 — ANSI escape codes and the executable bits

How colour works

A terminal is fundamentally a **character grid**. It displays bytes; it has no colour primitive, no framebuffer, no drawing API. Colour is an *illusion produced by convention*, inherited from the DEC VT100.

Any sequence starting with **ESC (0x1B)** is not displayed — the terminal **consumes** it and interprets it as a command that changes its internal state, then repaints. Like changing a printer setting.

```
ESC  [  <parameter bytes>  <final byte>
0x1B 0x5B                    e.g. 'm'   (SGR = Select Graphic Rendition)
```

Param	Meaning	Param	Meaning
`0`	reset everything	`30` – `37`	normal fg colours
`1`	bold / bright	`40` – `47`	normal bg
`2`	faint	`90` – `97`	bright fg
`7`	**reverse video**	`100` – `107`	bright bg
`22`	bold off	`39` / `49`	default fg / bg

Green is `\033[0;32m`

`0` (reset) `;` `32` (normal green) — then the text, then `\033[0m` to restore the default.

> **Consequence that matters:** escape sequences go to stdout like any other bytes. If stdout is a file or pipe, *nothing interprets them* and `ls > out.txt` would contain raw control garbage. That is why `use_color()` returns `isatty(STDOUT_FILENO)`. **Verified:** piped output had **0** escape bytes; on a real tty, **4**.

The executable bits

`st_mode` stores the execute bit **three times**, once per class of user, because a file can be executable by its owner but not by others.

Macro	Octal	Binary position	Meaning
`S_IXUSR`	`0100`	bit 6	owner may execute
`S_IXGRP`	`0010`	bit 3	group may execute
`S_IXOTH`	`0001`	bit 0	others may execute

```c
if (mode & (S_IXUSR | S_IXGRP | S_IXOTH))
    return ANSI_EXEC;        /* any one of them → executable, like GNU ls */
```

> The `|` combines the three bits into one mask; the single `&` tests them all at once.
>
> **Refinement:** setuid/setgid *mask* the execute character — a setuid binary prints `s` instead of `x`, and GNU `ls` correctly does not colour it green even though it is executable.

Q10 — Base cases, and why the full path must be rebuilt

The three base cases

A **base case** is the condition under which a recursive function returns **without calling itself**.

1. `opendir()` **fails** — the primary base case. > Recursion only continues into things already proven to be directories, > so every recursive call should succeed. This check is what makes > `do_ls()` safe when called directly from `main()` on a plain file.
2. `"."` **and** `".."` **are skipped** — the base case that actually terminates. > These are how a directory points at *itself* and at *its parent*. > Following `.` re-enters the same directory — infinite recursion at the > same level. Following `..` walks upwards forever. Once back-references > are removed from a **finite** tree, no cycle exists.
3. **A depth limit (64)** — the defensive backstop. > For bind-mounts and symlink chains that defeat the reasoning above. A > runaway recursion costs a **crash or a hang**, not merely wrong > output. Every recursive function should have one.

Resource behaviour: each call owns its frame and its `NameList`, so the stack grows with the tree's **depth**, not the file count. A wide directory is cheap; a deep one is not.

Why the full path must be rebuilt

```
snprintf(child, sizeof(child), "%s/%s", path, name);
```

Every system call resolves a relative path against the **process CWD** — not against the directory being listed. The CWD is wherever the user ran the command, and it never changes.

So inside `do_ls("parent_dir")`, `"subdir"` means `./subdir` — the directory the user typed the command in, **not** the child.

What actually happens if you call `do_ls("subdir")` from within `do_ls("parent_dir")`: - `opendir("subdir")` succeeds only **by accident**, if such a directory happens to exist in the CWD. - If it fails → `perror()` fires and the subtree is **silently dropped**. A bug that is easy to miss, because the top-level listing still looks correct. - If it succeeds → it lists the **wrong** directory, and its children are again resolved against the CWD. Running `./bin/ls -R a` against `a/b/c/d` would print header `a:`, list `a`'s children, then call `opendir("b")` — which does not exist in the root, so `ls: cannot open 'b'` and the recursion dies one level down.

> **The one-line rule:** always pass `current_path + "/" + entry_name`, so the path means the same thing at every depth. It is the same reason a shell's `cd` inside a recursive function must build an absolute path to return to where it started.

Two related safety points

- `snprintf()` bounds the copy **and its return value is checked**. A deep tree can exceed the buffer; truncating silently and then `lstat()`-ing the truncated path lists the wrong directory.
- `lstat()`**, not** `stat()`, for the type test — otherwise a symlink pointing back up the tree reports "directory" and we recurse forever.

Part 4 — Code walkthrough: every function, in order

Know the call flow cold. If the TA points at a function, you must say what it does in one sentence.

```
main()
  └── getopt("lxR")  →  display_mode, recursive
  └── do_ls(dir, depth)
        ├── printf("%s:\n", path)          ← the -R header
        ├── opendir / readdir loop         ← PHASE 1: read all names
        │     └── namelist_push()          ← strdup + realloc doubling
        ├── qsort(names, count, …, cmp_names)   ← PHASE 1b: sort ONCE
        ├── dispatch on display_mode       ← PHASE 2: print
        │     ├── print_long_entry()       ← -l
        │     ├── print_across()           ← -x
        │     └── print_columns()          ← default
        ├── recursion loop                 ← PHASE 3: descend
        │     └── do_ls(child, depth + 1)
        └── namelist_free()                ← BEFORE recursing
```

Function	One-sentence job
`main`	Parse options, loop over directory arguments, seed recursion at depth 0
`do_ls`	List one directory: read → sort → print → descend. **This is the recursive function**
`terminal_width`	Ask the kernel the terminal width via `ioctl(TIOCGWINSZ)`; fall back to 80
`namelist_init/push/free`	Growable array of strings: `realloc` doubling, `strdup` per name, matching `free`
`print_columns`	Down-then-across; the nested loop with `idx = c * num_rows + r`
`print_across`	Horizontal; `col = i % num_cols`, newline when `col == 0`
`print_long_entry`	One file per line: perms, nlink, user, group, size, time, name
`mode_to_perms`	`st_mode` → the ten-character `rwxr-xr-x` string, incl. special bits
`format_mtime`	`ctime()` → `"Mar  5 14:30"` (recent) or `"Mar  5 2024"` (older)
`cmp_names`	`qsort` comparator: double-dereference + `strcmp`
`color_for_mode`	Decide the escape sequence from `st_mode` (and the name for archives)
`print_colored_padded`	Print a name in colour; pad **only the bare name** so columns align
`use_color`	`isatty(STDOUT_FILENO)` — suppress escapes when redirected
`has_archive_ext`	`strrchr` for the last dot, then exact tail match

Part 5 — "Explain this line" traps

The TA will point at something. Here are the likely picks and the answers.

`if (mode & (1 << (8 - i)))` > Bit `8` is "owner read" and maps to `rwx[0]`; bit `0` is "other execute" > and maps to `rwx[8]`. Hence `8 - i`. The AND is a one-bit mask, so it is > a boolean test.

`num_rows = (count + num_cols - 1) / num_cols` > **Ceiling division.** `(a + b - 1) / b` rounds `a/b` up. Without the `+ b - 1` > it would truncate and we would lose the last row.

`printf("%-*s", col_width, name)` > `*` takes the width from the argument list; `-` left-aligns. `col_width` is > `int`, so the cast `(int)` is needed because `*` expects an `int`, not a > `size_t`.

`const char *sa = *(const char *const *)a;` > `qsort` passes element **addresses**. Our elements are `char *`, so `a` is > a `char **` — one dereference to reach the slot, one to reach the string.

`if (snprintf(...) >= (int)sizeof(child))` > `snprintf` returns the length it **would have written**, so `>=` the buffer > size means truncation happened. Truncating silently and then `lstat()`-ing > the truncated path would list the wrong directory.

`nl.names[nl.count] = strdup(name);` > `readdir` reuses one internal buffer, so the raw `d_name` pointer would be > invalidated by the next call. `strdup` copies onto the heap, so each slot > owns an independent string — and every `strdup` needs a matching `free`.

`if (nl.count == nl.capacity) { ... realloc ... }` > The array grows **geometrically** — doubling — so total reallocation work > is amortised O(1) per element instead of O(n). Without the check we would > `realloc` on every single insert.

`if (tmp == NULL) { perror(...); exit(...); }` > `realloc` on failure returns `NULL` but **leaves the original block valid > and still owned by us**. So we must not overwrite `nl.names`, or we would > leak it.

`errno = 0;` **before the readdir loop** > `readdir` returns `NULL` both at end-of-directory **and** on error. Setting > `errno = 0` first is the only way to tell the two apart afterwards.

`lstat(path, &st) != 0` **instead of** `stat` > Do not follow symlinks, so a link reports as a link, and we do not > recurse into a symlinked directory.

`getopt(argc, (char *const *)argv, "lxR")` > The cast is **required, not cosmetic**. The base declares > `int main(int, char const *argv[])`, making `argv` a `const char **`, but > `getopt` is declared `int getopt(int, char *const *, const char *)`. > Those are *incompatible pointer types* and GCC rejects it. The cast is safe > because `getopt` may **permute** `argv` and substitute option values, but > never modifies the strings.

Part 6 — The three bugs you found

This is your strongest material. Proactively offering it shows you tested rather than guessed. If the TA asks "how do you know it works?", say:

> "I ran it under AddressSanitizer, UndefinedBehaviorSanitizer and LeakSanitizer — 75 runs, including symlink cycles, broken links, FIFOs, names with spaces, empty directories and unreadable paths. It found three real bugs, which I fixed."

Bug 1 — `qsort(NULL, 0, ...)` on an empty directory

> `qsort`'s first parameter is declared a pointer that *"shall not be null"*, so sorting zero elements through `NULL` is formally undefined behaviour even though glibc tolerates it. UBSan reported *"null pointer passed as argument 1, which is declared to never be null"*. **Fix:** guard with `if (count > 1)` — sorting 0 or 1 element is a no-op. Testing `count > 1` also guarantees the array is non-`NULL`, since `namelist_push` is what allocates it.

Bug 2 — Column misalignment in `-x` (`alpha.txtsub2`)

> I first implemented `-x` with a running cursor `used`, as the brief describes. It glued two names together. **The cause:** a *padded* item occupies the full `col_width` on screen, but an *unpadded* item occupies only its own length. Counting the padded case correctly and the unpadded case only by `len()` leaves `used` no longer a multiple of `col_width`, so every later wrap test is off by the missing padding. **Fix:** derive the column from the index — `col = i % num_cols`. That removes the entire class of bug, because the item-to-position mapping becomes a pure function of the index and can never drift. **Why vertical mode was immune:** every item there *is* padded to `col_width`, so `used` stays a clean multiple.

Bug 3 — Trailing whitespace

> Padding the final item of a row left visible spaces at end of line, which real `ls` never emits and which show as noise in `ls | cat -A`. **Fix:** pad only when a real item follows on that row.

Part 7 — Memory and safety Q&A

"What happens if `malloc` fails?" > `namelist_push` calls `perror("realloc")` then `exit(EXIT_FAILURE)`. It > must **not** overwrite `nl.names` first, because on failure `realloc` > returns `NULL` while leaving the original block valid and still owned by > us — overwriting would leak it.

"How do you avoid leaking under `-R`?" > `namelist_free(&nl)` runs **before** the recursion loop, not after. So at > most one level's worth of entries is live at any moment. If it ran at the > end, every level of the tree would hold its own array simultaneously.

"Why `snprintf` everywhere and never `sprintf`?" > `sprintf` has no bound — an unbounded write into a fixed buffer is a > textbook stack buffer overflow. A deep tree or a very long filename would > overrun `char full[4096]`.

"Why do you check `snprintf`'s return value?" > It returns the length it *would have* written. `>=` the buffer size means > truncation happened, and we must not then `lstat` a truncated path.

"Is `getpwuid()`'s result safe to hold?" > Not across calls — it is a static buffer the next call overwrites. Never > `free()` it. We print immediately, so it is safe here; if we had to keep > it we would `strdup` it.

Part 8 — Git workflow questions

"Why a feature branch per feature?" > So each feature is isolated and reviewable, and `main` always stays > buildable. It also makes the graded history legible — the TA can see each > feature developed independently rather than appearing all at once.

"Why `--no-ff` on the merge?" > It forces a real merge commit even when fast-forward is possible, so the > graph records *that a feature was developed on its own branch and merged*, > rather than looking like linear development. A fast-forward would hide > that history.

"Annotated tag or lightweight?" > Annotated (`git tag -a`). A lightweight tag is just a pointer to a commit; > an annotated tag is a full object with a tagger, date and message — which > is what makes the GitHub Releases page show a description.

"Why push all branches at the end?" > The brief requires it. The feature branches are the evidence of the > incremental process; `main` alone would hide it.

"What does `bin/` in the repo contain?" > The compiled `ls`. The brief explicitly lists `bin/` among the directories > to stage, and requires the binary as a release asset.

Part 9 — Demonstrating it live

If asked to run it, these are the five commands that show the most.

```
make                          # builds src/ls-v1.0.0.c -> bin/ls
./bin/ls                      # default: down-then-across columns
./bin/ls -l                   # long listing
./bin/ls -x                   # horizontal columns
./bin/ls -R src               # recursive
./bin/ls -lR src              # flags combine

# rebuild any tagged version
make SRC=versions/ls-v1.2.0.c

# proof that width detection works — resize the terminal and re-run
# proof that colour is suppressed when redirected
./bin/ls | cat -A             # no ESC sequences
```

Things to say while it runs: - "Watch — resize the window and run it again; the column count follows." - "That's piped, so `isatty` is false and the output is clean text." - "This is `-x`, so left to right; without it, down then across." - "`-R` uses `lstat` before descending, so it does not follow a symlink back into the tree."

Part 10 — Limitations you should volunteer

Naming your own limitations reads as mastery, not weakness. If the TA finds one you did not mention, it looks like you did not test.

1. `ls -l -x` lets the last flag win; GNU `ls` rejects the combination.
2. **Column width is uniform**, from the longest filename — as the brief specifies. GNU `ls` optimises **per-column** widths and can fit noticeably more columns.
3. No `-a`, `-t`, `-S`, `-h`, `-1`, `-r`, `-F`, `-i`, `-n`, `-d`, `-G`.

4. The `path:` header is printed unconditionally; real `ls` prints it only under `-R`.
5. Owner/group columns are fixed at 8 characters; GNU `ls` sizes them to the widest actual name.
6. `getpwuid` / `getgrgid` results are not cached, so `-l` over a large directory does one `/etc/passwd` lookup per file.
7. No Makefile shipped with the starter — Feature-1 required writing one from scratch.

Part 11 — Rapid-fire flashcards

Cover the answers. Say them out loud.

Q: Why `lstat` and not `stat`? A: Follow vs do not follow. `ls -l` must show `l` and the `-> target`, which exist only in the link's own inode. Under `-R` it also stops symlink cycles.

Q: What is `S_IFMT`? A: The mask `0170000` covering all four file-type bits, used so the equality test is not polluted by permission bits.

Q: `0100` in octal — which bit is that? A: `S_IXUSR`, owner execute. The execute bits are `0100` / `0010` / `0001`.

Q: Why lowercase `s` and uppercase `S`? A: Lower case means the execute bit is *also* set; upper case means it is not.

Q: How does `ls` know the terminal width? A: `ioctl(STDOUT_FILENO, TIOCGWINSZ, &ws)`, then `ws.ws_col`.

Q: Why not `system("tput cols")`? A: It forks a shell and execs a process on every invocation.

Q: Formula for the number of columns? A: `num_cols = width / (longest_name + 2)`.

Q: Formula for the index in down-then-across? A: `c * num_rows + r`.

Q: Why must there be a nested loop there? A: Down-then-across is a transposition; the index-to-position map is not linear.

Q: What does `qsort` pass to the comparator? A: The **addresses** of two elements. Ours are `char *`, so the cast happens twice.

Q: Why `const void *`? A: `void *` because `qsort` is type-agnostic; `const` because it sorts in place but never modifies elements.

Q: Why never return `(a - b)` from the comparator? A: Arithmetic on unrelated addresses — undefined behaviour. `strcmp` already returns the right value.

Q: The escape sequence for green? A: `\033[0;32m`, reset with `\033[0m`.

Q: Why gate colour on `isatty`? A: Escape codes are ordinary bytes; when redirected nothing interprets them, so the file would contain control garbage.

Q: Why pad the bare name and emit the reset *after*? A: Escape codes are invisible but count as bytes, so padding a pre-coloured string would mis-pad every column by the length of the escape sequence.

Q: What is the base case of our recursion? A: Three layers: `opendir` fails; `.` and `..` are skipped; depth limit.

Q: Why must we rebuild the full path when recursing? A: Relative paths resolve against the process CWD, not the directory being listed. `"subdir"` would mean `./subdir`.

Q: Why is the sort applied in `do_ls` and not in each display function? A: Once, centrally — so every display mode is sorted and none of them needs to know sorting exists.

Q: How do you prove it works? A: Zero warnings under `-Wall -Wextra`; 75 runs under ASan/UBSan/LSan with zero findings; output compared against the real `ls`; column count verified to track terminal width 40→200; stress test with 3200 files at 9 nesting levels.

Part 12 — Last-minute checklist

- [ ] The 60-second pitch, without reading
- [ ] `stat` vs `lstat` — *both* the `-l` and the `-R` reason
- [ ] `S_IFMT`, the nine permission bits, `s`/`S` and `t`/`T`
- [ ] `c * num_rows + r` and why one loop fails
- [ ] `qsort` passes addresses; the cast happens **twice**
- [ ] `\033[0;32m`; why `isatty`
- [ ] The three base cases
- [ ] Why the full path must be rebuilt in recursion
- [ ] The three bugs, with causes — **volunteer these**
- [ ] `getopt` cast and why it is required
- [ ] Two or three limitations you can name yourself

> **Final reminder.** Every answer in this guide was written so you can *say it*, not just read it. If the TA asks a question that is not here, the honest and correct answer is *"I did not implement that, here is my reasoning for how I would"* — followed by the reasoning. That scores far better than a guess. And a wrong confident answer is worse than an honest one: the brief makes demonstration of understanding the pass/fail condition for the whole assignment.

Prepared for BSDSF24A040-OS-A02 · OS Programming Assignment 02 · 100 marks